

Item-Difficulty-Aware Learning Path Recommendation

From a Real Walking Perspective

Zhang, Shen, Xu, Huang, Wu, Sha & Wang

University of Science and Technology of China · iFLYTEK AI Research

Presentado por: Ignacio Sanhuesa y David Leal





Motivación

¿Qué pasa cuando aprender se parece a caminar de noche en terreno irregular?

Problema 1

Camino Rugoso

La dificultad sube y baja sin control:

- Pasa de ejercicios fáciles a muy difíciles sin transición
- Frustra al estudiante y baja su motivación
- Como caminar en la oscuridad en terreno irregular

Problema 2

Camino Ineficiente

Mismo esfuerzo sin importar la dificultad:

- Muy pocos intentos en ítems difíciles antes de cambiar
- Demasiados intentos en ítems imposibles sin avanzar
- Desperdicio de tiempo del estudiante

Causa raíz: los métodos existentes ignoran la dificultad de los ítems

Definición Formal del Problema

Recomendación de camino de aprendizaje paso a paso (step-by-step)

LI

Items de aprendizaje

conceptos o habilidades: ej. multiplicacion, algebra

PI

Items de practica

ejercicios o preguntas: ej. $4 \times 9 = ?$, resolver $2x - 5 = 3$

G

Objetivos de aprendizaje del estudiante

las metas que debe alcanzar en la sesion

H

Historial de aprendizaje (item, score)

pares (e, score) acumulados en tiempo real

Objetivo a maximizar:

$$E_p = \frac{E_e - E_s}{E_{sup} - E_s}$$

E_p : Efectividad del aprendizaje

E_e : Puntaje final (post-sesion)

E_s : Puntaje inicial (pre-sesion)

E_{sup} : Puntaje maximo posible

Sesion: score inicial $\rightarrow$ etapas (L-Item) $\rightarrow$ pasos (P-Item) $\rightarrow$ score final $\rightarrow$ calcular E_p

Estado del Arte

Metodos existentes de Learning Path Recommendation (LPR)

Generacion Completa

- Se genera el camino completo de una vez
- Decision trees, RNN, set-to-sequence (SRC)
- SRC: ranking-based + encoding/decoding
- Desventaja: ignora cambios cognitivos del estudiante

Generacion Paso a Paso

- Camino generado con feedback en tiempo real
- CSEAL: actor-critic con navegacion cognitiva
- GEHRL: Hierarchical RL para LPR
- Desventaja: ninguno considera la dificultad de items

Gap: ningun metodo controla adecuadamente la dificultad para garantizar caminos suaves y eficientes

Contribuciones del Paper

DLPR: Difficulty-constrained Learning Path Recommendation

01

Analisis desde perspectiva real

Identifican y formalizan el problema de caminos rugosos e ineficientes — ningun trabajo previo lo analizaba desde la perspectiva de la dificultad.

02

Grafo Jerarquico (HG + HGNN)

Separan items de aprendizaje y practica, integrando dificultad en las representaciones via Hierarchical Graph Neural Network.

03

Framework DHRL

Difficulty-driven Hierarchical Reinforcement Learning con L-Agent (conceptos) y P-Agent (ejercicios) con comunicacion bidireccional.

04

State-of-the-art en 3 simuladores

Experimentos en 3 simuladores basados en 2 datasets benchmark demuestran mejoras consistentes en efectividad de aprendizaje.

Marco Teorico

Conceptos necesarios para entender DLPR

Knowledge Tracing (KT)

Rastrea el estado de conocimiento del estudiante en tiempo real. DLPR usa DIMKT (Difficulty Matching KT), que además considera la dificultad del ítem al modelar el cambio cognitivo.

Graph Neural Networks (GNN)

Agrega información estructural en grafos. HGNN propaga características entre conceptos y ejercicios en el grafo jerárquico, capturando relaciones de alto orden.

Hierarchical Reinforcement Learning

Dos agentes anidados: alto nivel (L-Agent, elige conceptos) y bajo nivel (P-Agent, elige ejercicios). Permite decisiones a distintas granularidades con comunicación bidireccional.

Markov Decision Process (MDP)

Formaliza la recomendación como problema de decisión secuencial: estado (conocimiento del estudiante) -> acción (ítem recomendado) -> recompensa (mejora de score).

Solución: Framework DLPR

Dos módulos principales que se complementan



Modulo 1: Grafo Jerarquico

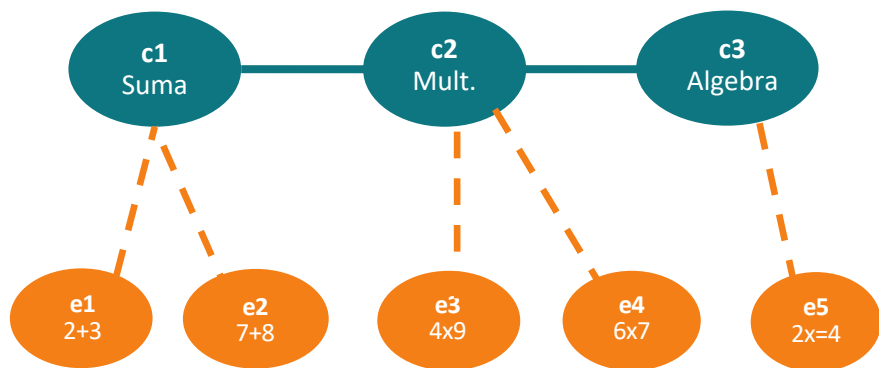
- Separa items de aprendizaje (LI) y practica (PI)
- Edges sucesivos entre conceptos (LI->LI)
- Edges evaluativos (PI->LI que evalua)
- HGNN agrega info jerarquica con dificultad
- Genera embeddings enriquecidos $\tilde{X}$ P para DIMKT

Modulo 2: DHRL Framework

- L-Agent (alto nivel): selecciona conceptos
- P-Agent (bajo nivel): selecciona ejercicios
- Communication Mechanism (bidireccional)
- Knowledge State via DIMKT enriquecido
- Controla suavidad y eficiencia del camino

Detalle: Grafo Jerarquico + HGNN

Modulo 1: Representaciones enriquecidas con dificultad



- Learning Item
- Practice Item
- Relacion sucesiva
- Relacion evaluativa

Dificultad del ítem de práctica:

$$DP_j = \Sigma (a_i=0) / |S_j| \times \lambda_p$$

Embedding con dificultad:

$$\mathbf{X}_j^p = \mathbf{W}_p^T [\mathbf{E}_p \oplus \mathbf{E}_{dp}] + \mathbf{b}_p$$

Agregacion HGNN (mean pooling):

$$\tilde{\mathbf{X}}_j^p = \mathbf{X}_j^p \oplus (1/|N_j|) \Sigma \mathbf{X}_i^L$$

$\tilde{\mathbf{X}}_j^p$ reemplaza el embedding original en DIMKT para rastrear conocimiento considerando dificultad

Detalle: Framework DHRL

Modulo 2: Dos agentes que se comunican para generar el camino

L-Agent (Alto Nivel)

Selecciona que **CONCEPTO** estudiar a continuacion

- Estado: conocimiento actual ht + metas G
- Accion: escoger proximo learning item ci
- Espacio adaptativo (Reference Path):
combina camino A* con vecinos
evita atajos miopes al objetivo
- Recompensa: mejora en score del estudiante



Comm.
Mech.

P-Agent (Bajo Nivel)

Selecciona que **EJERCICIO** practicar

- Estado: conocimiento ht + concepto actual ci
- Accion: escoger ejercicio eji para practicar ci
- Control de suavidad del camino:
mantiene dificultad cercana al nivel del estudiante
usa info del L-Agent via Communication Mech.
- Limita intentos segun dificultad (eficiencia)

Diferencia vs GEHRL: comunicacion BIDIRECCIONAL entre agentes + espacios de accion distintos por nivel

Evaluación

Configuración experimental

Datasets

- Junyi: ejercicios de matematicas (USTC)
- ASSIST09: dataset publico de educacion

Simuladores (3)

- KSS (Basado en estructura de conocimiento)
- KES-Junyi (Basado en evolución de conocimiento)
- KES-ASSIST09

Baselines comparados

DQN

RL basico
sin jerarquia

CSEAL

Actor-critic
cognitivo

GEHRL

HRL para LPR
sin dificultad

KNN

Caminos más
similares

Metrica principal: $E_p = (E_e - E_s) / (E_{sup} - E_s)$ — mayor es mejor (mas aprendizaje)

Resultados

DLPR logra estado del arte en los 3 simuladores y 2 datasets

		KNN	GRU4Rec	DQN	Actor-Critic	CB	RLTutor	CSEAL	GEHRL	DLPR
KSS	Learning-Steps	54	72	48	41	56	34	29	14	8
	Cog-Gap	0.3216	0.3889	0.3709	0.3921	0.3514	0.3336	0.3639	0.2436	0.1017
	Diff-MAD	0.3886	0.3267	0.3574	0.3749	0.3309	0.3696	0.3231	0.2704	0.1200
KES-Junyi	Learning-Steps	-	-	-	-	219	-	128	96	33
	Cog-Gap	-	-	-	-	0.3413	-	0.3898	0.3641	0.1313
	Diff-MAD	-	-	-	-	0.3687	-	0.2791	0.3144	0.1614
KES-ASSIST09	Learning-Steps	-	-	-	-	-	536	378	185	69
	Cog-Gap	-	-	-	-	-	0.4197	0.3823	0.3562	0.1391
	Diff-MAD	-	-	-	-	-	0.2947	0.2893	0.2998	0.1577

+57%

sobre GEHRL en pasos
(mejor baseline)

9/9

configs donde
DLPR es #1

3

simuladores
validados

* Mejora estadísticamente significativa ($p < 0.05$)

Conclusiones



Problema real identificado

Caminos rugosos e ineficientes son causados por ignorar la dificultad — ningún trabajo previo lo analizaba desde esta perspectiva.



DLPR resuelve ambos problemas

El grafo jerárquico + DHRL produce caminos más suaves (controlando dificultad) y más eficientes (calibrando intentos por dificultad).



Estado del arte demostrado

Mejoras consistentes en 9 combinaciones dataset x simulador, validando que la dificultad es una señal crítica en LPR.



Trabajo futuro

Explorar más tipos de feedback (tiempo de respuesta, confianza), y extender a dominios fuera de matemáticas.

Referencias

Bibliografía citada en esta presentación

- [1] Zhang, Shen, Xu, Huang, Wu, Sha & Wang (2024). *Item-Difficulty-Aware Learning Path Recommendation: From a Real Walking Perspective*. En Proc. KDD '24 (pp. 4167–4178). <https://doi.org/10.1145/3637528.3671947> — *paper principal*
- [2] Liu et al. (2019). *Exploiting Cognitive Structure for Adaptive Learning*. En Proc. KDD '19 (pp. 627–635). — *baseline CSEAL*
- [3] Li et al. (2021). *GEHRL: Goal-conditioned Efficient Hierarchical Reinforcement Learning for Learning Path Recommendation*. — *baseline principal*
- [4] Chen et al. (2023). *Set-to-Sequence Ranking-Based Concept-Aware Learning Path Recommendation*. En Proc. AAAI '23. — *baseline SRC*
- [5] Shen et al. (2022). *DIMKT: Difficulty Matching Knowledge Tracing*. En Proc. KDD '22. — *módulo de knowledge tracing*
- [6] Hamilton, Ying & Leskovec (2017). *Inductive Representation Learning on Large Graphs*. En NeurIPS '17. — *base de la arquitectura HGNN*

Material Adicional

Formulas detalladas | Metricas | Pseudocodigo | Simuladores

[Extra] Metricas de Evaluacion

Metricas principales y secundarias reportadas en el paper

Learning Effectiveness (Ep)

$$Ep = (Ee - Es) / (Esup - Es)$$

Metrica principal. Mide la ganancia normalizada de conocimiento entre el test inicial y el test final de la sesion. Valores en [0, 1]. Mayor = mejor aprendizaje.

Path Length (PL)

$$PL = \text{total steps in path } P$$

Numero total de pasos (ejercicios presentados) en el camino. Caminos mas cortos con mismo Ep indican mayor eficiencia. Se reporta junto a Ep.

Difficulty Smoothness (DS)

$$DS = (1/T) \times \sum |diff(e_t) - diff(e_{t-1})|$$

Promedio de la variacion absoluta de dificultad entre ejercicios consecutivos. Valores mas bajos = camino mas suave. Complementa a Ep para medir calidad.

Goal Achievement Rate (GAR)

$$GAR = |goals\ achieved| / |G|$$

Proporcion de objetivos de aprendizaje alcanzados (score >= umbral) al final de la sesion. Mide si el camino fue suficientemente completo.

[Extra] Pseudocodigo DHRL

Algoritmo de generacion del camino de aprendizaje

Input: learner, goals G, HG graph

Output: learning path P

```
1. h0 = DIMKT.init(learner.history)
2. FOR each stage i:
3.   sl_i = [h_{i-1}, G]
4.   c_i = L-Agent.select(sl_i, A_L)
5.   msg_L = L-Agent.message(c_i, h_{i-1})
6.   attempts = 0
7.   WHILE not mastered(c_i):
8.     sp_j = [h_t, embed(c_i)]
9.     e_j = P-Agent.select(sp_j, A_P)
10.    score = simulate(learner, e_j)
11.    h_t = DIMKT.update(h_{t-1}, e_j, score)
12.    P.append(e_j)
13.    attempts += 1
14.    IF attempts >= max(c_i, msg_L): BREAK
15.  msg_P = P-Agent.message(h_t, attempts)
16.  L-Agent.receive(msg_P)
17.  IF all_goals_reached(G): BREAK
18. RETURN P
```

Inicializar estado
de conocimiento con DIMKT

L-Agent elige concepto
y envia mensaje al P-Agent

P-Agent elige ejercicio
controlando dificultad

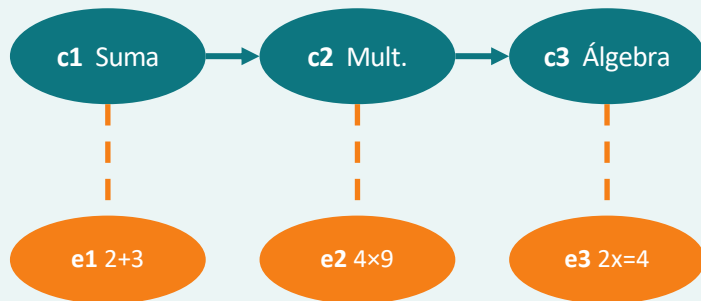
Limite de intentos
por dificultad (eficiencia)

Comunicacion
bidireccional

[Extra] Fórmulas: Grafo + Embeddings

Módulo 1 · Cómo se estructura el grafo y se inyecta la dificultad en los embeddings (Ec. 3-4)

Grafo Jerárquico (HG)



- Learning item (LI)
- Practice item (PI)
- Sucesiva (LI → LI)
- - Evaluativa (PI → LI)

Embedding del concepto (Ec. 3)

$$\mathbf{X}_{\mathbf{i}}^{\mathbf{L}} = \mathbf{W}_{\mathbf{L}}^{\mathbf{T}} [\mathbf{E}_{\mathbf{l}} \oplus \mathbf{E}_{\mathbf{dl}}] + \mathbf{b}_{\mathbf{L}}$$

Embedding del ejercicio (Ec. 4)

$$\mathbf{X}_{\mathbf{j}}^{\mathbf{P}} = \mathbf{W}_{\mathbf{P}}^{\mathbf{T}} [\mathbf{E}_{\mathbf{p}} \oplus \mathbf{E}_{\mathbf{dp}}] + \mathbf{b}_{\mathbf{P}}$$

$\mathbf{E}_{\mathbf{l}} / \mathbf{E}_{\mathbf{p}}$ embedding base (identidad del ítem)

$\mathbf{E}_{\mathbf{dl}} / \mathbf{E}_{\mathbf{dp}}$ embedding paramétrico de la dificultad

$\oplus$ concatenación: identidad y dificultad se unen

La dificultad se inyecta desde el embedding mismo: cada ítem 'conoce' su nivel de dificultad antes de propagarse por el grafo.

[Extra] Fórmula: Dificultad del Ítem (DP_j)

Módulo 1 · Cómo se cuantifica la dificultad empírica de cada ítem de práctica (Ec. 2)

Dificultad calculada para el ítem de práctica j.

Filtro de fallos: respuestas incorrectas de los alumnos.

$$DP_j = \frac{\sum (a_i = 0)}{|S_j|} \times \lambda_p$$

Total de alumnos que intentaron el ejercicio.

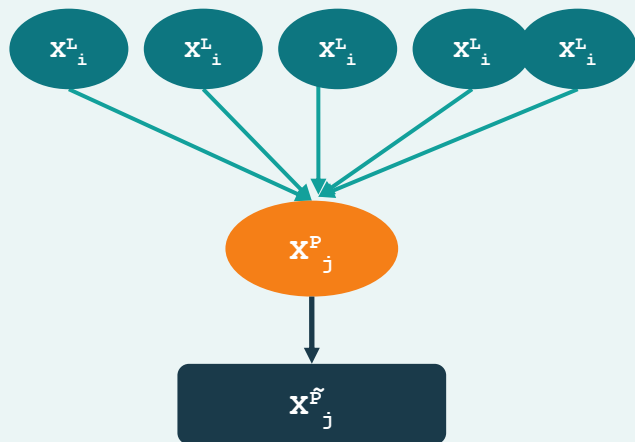
Niveles de dificultad predefinidos (p. ej. 50 estratos).

Insight: es la tasa de fallo empírica escalada a un nivel de dificultad absoluto.

[Extra] Fórmula: Agregación HGNN

Módulo 1 · El embedding del ejercicio absorbe el peso cognitivo de sus conceptos vecinos (Ec. 5)

Mean pooling de vecinos



$$\mathbf{x}_j^{\hat{P}} = \mathbf{x}_j^P \oplus (1/|N_j|) \sum \mathbf{x}_i^L$$

Hierarchical Graph Neural Network · Ec. 5

$\mathbf{x}_j^P$ embedding propio del ejercicio (con su dificultad).
 $(1/|N_j|) \sum \mathbf{x}_i^L$ promedio de los conceptos vecinos N_j .

El embedding final del ejercicio es consciente no solo de su propia dificultad, sino del peso cognitivo de todos los conceptos que evalúa.

[Extra] Fórmulas: Recompensa L-Agent

Módulo 2 · Alto nivel · Política actor-crítico y señal de recompensa por efectividad (Ec. 7-9)

Actor — Política (PPO)

$$c_i = \text{Softmax} (\text{FC} (s_i^1))$$

Emite la probabilidad de seleccionar el concepto c_i dentro del espacio de acción adaptativo.

Crítico — Valor

$$V_L (s_i^1 ; \phi_L) = \text{FC} (s_i^1)$$

Estima el retorno esperado desde el estado actual. Se entrena con error cuadrático medio (MSE).

Señal de recompensa (Ec. 9)

$$r_i^1 = E_p \quad (\text{solo en la última etapa})$$

La recompensa es puramente la efectividad del aprendizaje: la mejora real en los exámenes. Prioriza el éxito a largo plazo.

Entrenamiento: PPO-clip para el actor · MSE para el crítico — igual que el baseline GEHRL.

[Extra] Fórmulas: Recompensa P-Agent

Módulo 2 · Bajo nivel · Suavidad de dificultad + ganancia cognitiva, balanceadas (Ec. 12-14)

Recompensa por suavidad (Ec. 12)

$$r_t^{p1} = - (DP_t - DP_{t-1})^2$$

Penaliza cuadráticamente los saltos de dificultad entre ejercicios consecutivos: castiga los cambios erráticos → camino suave.

Recompensa por dominio (Ec. 13)

$$r_t^{p2} = h_i - h_{i-1} \quad \text{si} \quad h_i > \text{Thre}$$

Mide el delta real en el estado de conocimiento (provisto por DIMKT).
Recompensa el avance cognitivo del estudiante.

Recompensa unificada (Ec. 14)

$$r_t^p = \alpha_1 \cdot r_t^{p1} + \alpha_2 \cdot r_t^{p2}$$

Balancea suavidad vs. dominio. En los experimentos $\alpha_1 = \alpha_2 = 0.5$.

El P-Agent equilibra mantener al estudiante en una zona de confort de dificultad **y obligarlo a cruzar el umbral del conocimiento.**

[Extra] Fórmula: Efectividad (E_p)

La métrica objetivo que el sistema entero maximiza (Ec. 1)

Efectividad del aprendizaje —
ganancia normalizada $\in [0, 1]$.

E_e : puntaje final, tras el camino (post-
sesión).

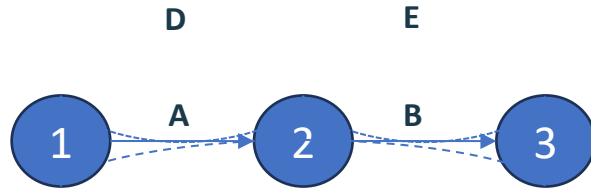
$$E_p = \frac{E_e - E_s}{E_{sup} - E_s}$$

E_{sup} : puntaje máximo posible (= nº de
objetivos).

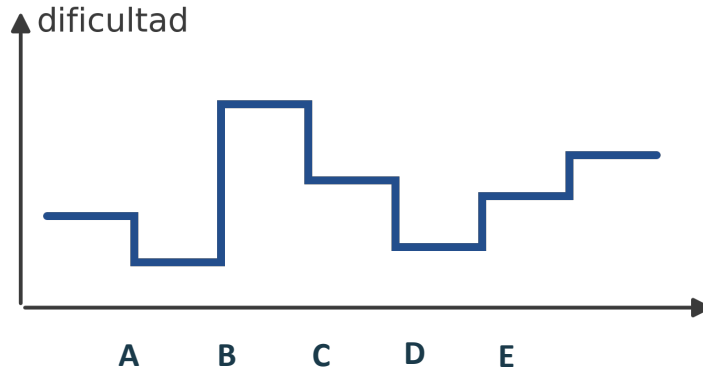
E_s : puntaje inicial, antes de empezar
(pre-sesión).

Conexión clave: es exactamente la recompensa del L-Agent ($r_l = E_p$) — todo el sistema optimiza la ganancia real de conocimiento.

[Extra] Caminos con/sin dificultad

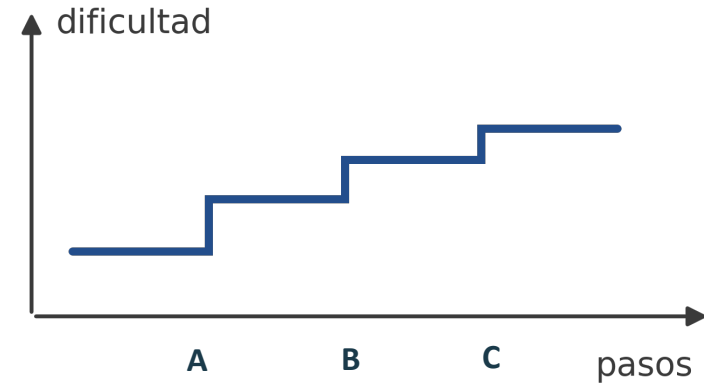


C



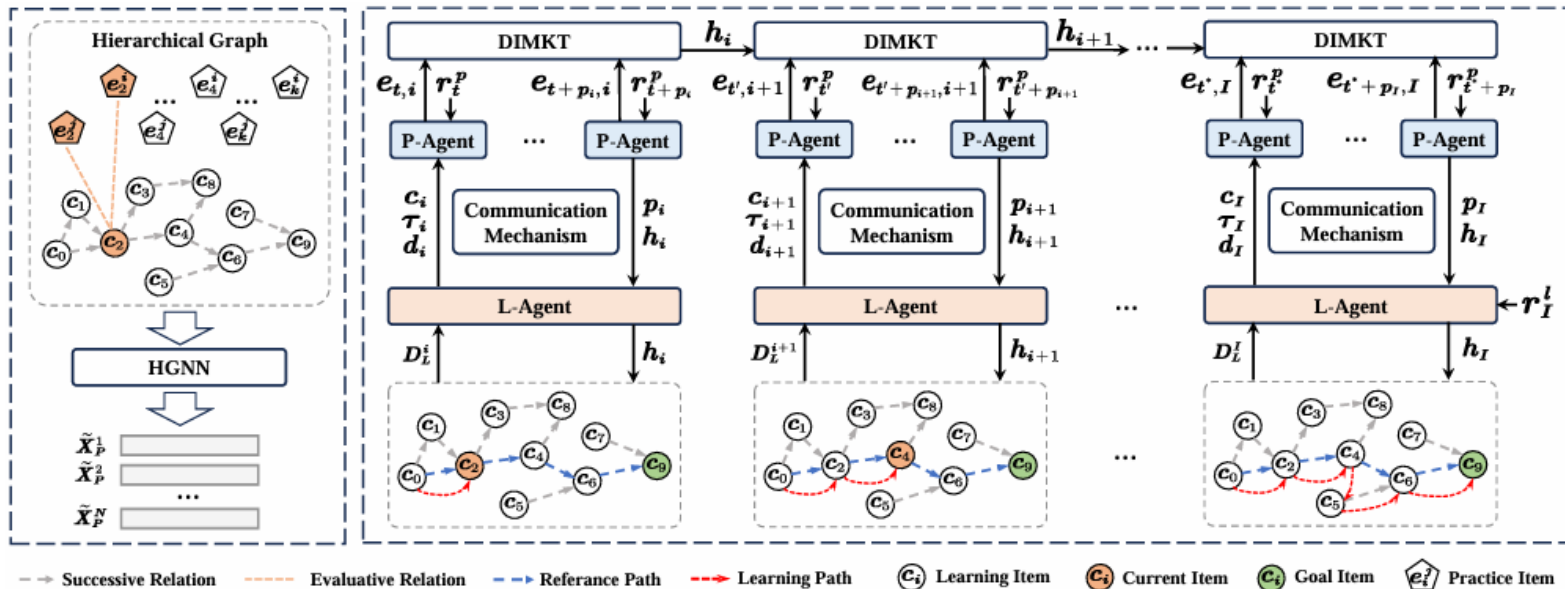
Sin considerar dificultad

1. Adición
2. Multiplicación
3. Ecuación algebraica



Considerando dificultad

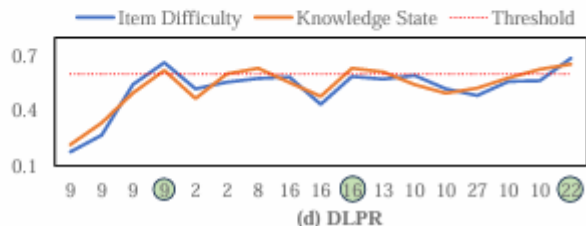
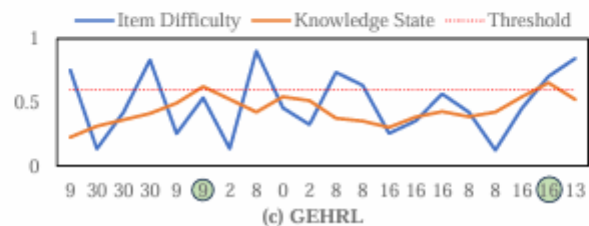
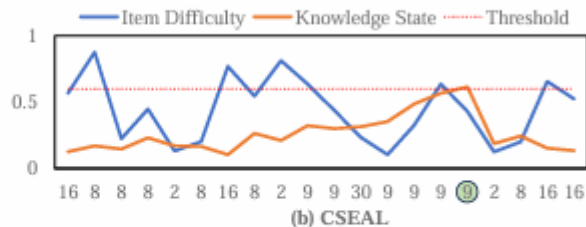
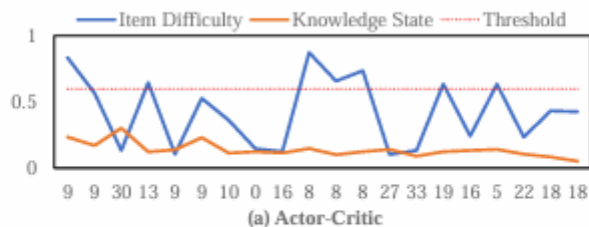
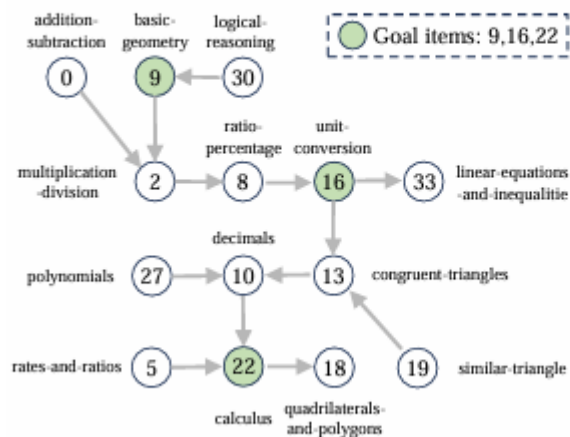
[Extra] Visión general del framework DLPR



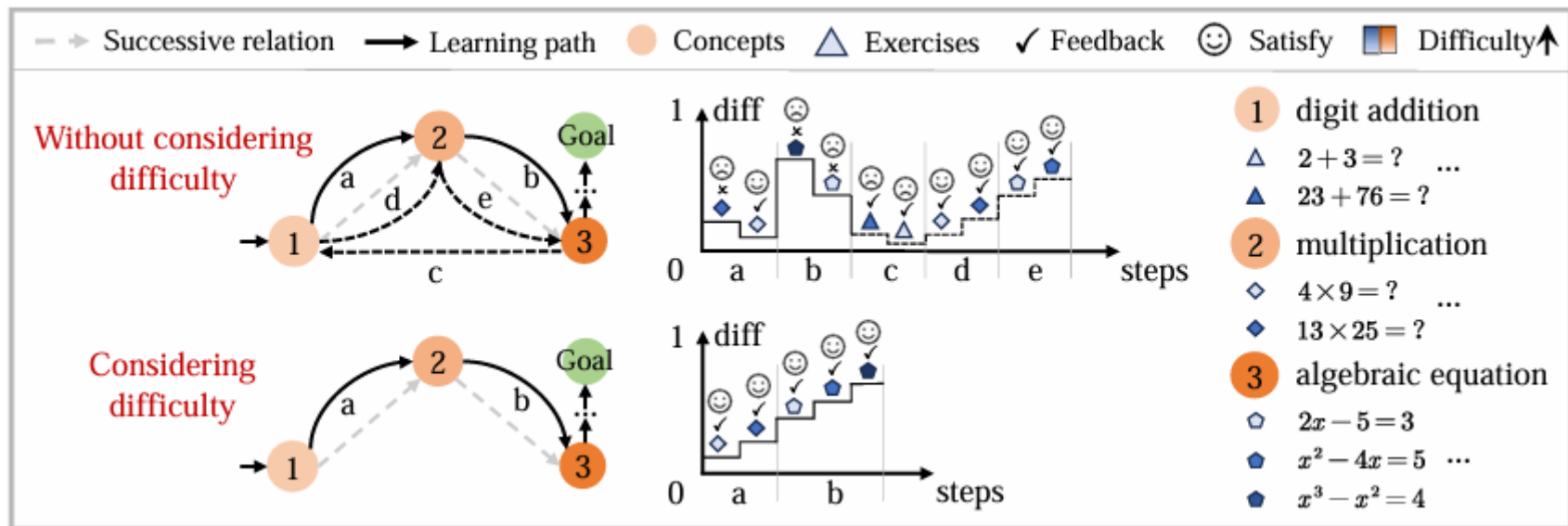
[Extra] Comparación de caminos aprendizaje

Item-Difficulty-Aware Learning Path Recommendation: From a Real Walking Perspective

KDD '24, August 25–29, 2024, Barcelona, Spain



[Extra] Caminos con/sin dificultad



[Extra] Desempeño según segmentación

